

A Neural Dependency Parser from Scratch

Comprehensive Architecture, Mathematical Formulations, Dual Cross-Entropy Loss, Structural Masking, Greedy Tree Decoding, and Cycle-Breaking Graph Repair.

COURSE	TECH STACK	ARCHITECTURE	MODEL PARAMETERS
Natural Language Processing	Python 3.13, PyTorch 2.9 (CPU), NumPy	Embedding → BiLSTM → Bilinear Arc/MLPs	176,936 (~715 KB Checkpoint on Disk)

176,936

TRAINABLE PARAMETERS

100.0%

HELD-OUT UAS / LAS

~25s

TRAINING TIME (CPU)

0

PRETRAINED / BLACKBOX LIBS

1. Introduction & Theoretical Foundations

Dependency parsing is the core NLP task of discovering the grammatical backbone of a sentence: determining which word depends syntactically on which other word (its *head* or *governor*), and identifying the precise grammatical type of that dependency relation (e.g., nominal subject, direct object, determiner, adjectival modifier).

Unlike **phrase-structure (constituency) parsing**, which builds deeply nested hierarchies of abstract phrasal constituents (NP, VP, PP) with non-terminal nodes, a dependency tree connects words **directly to other words** via typed directed arrows. For a sentence of n words, a valid dependency tree consists of exactly n directed edges, connecting every word to its unique head (including an artificial sentinel root node at index 0). This representation is language-neutral, computationally compact, and maps directly to predicate-argument semantic extraction.

CONSTITUENCY GRAMMAR VS. DEPENDENCY GRAMMAR

Constituency Parsing: Recursive grouping into phrasal constituents (e.g., [S [NP The dog] [VP chased [NP the cat]]]). Requires many non-terminal nodes; sensitive to word-order variations.

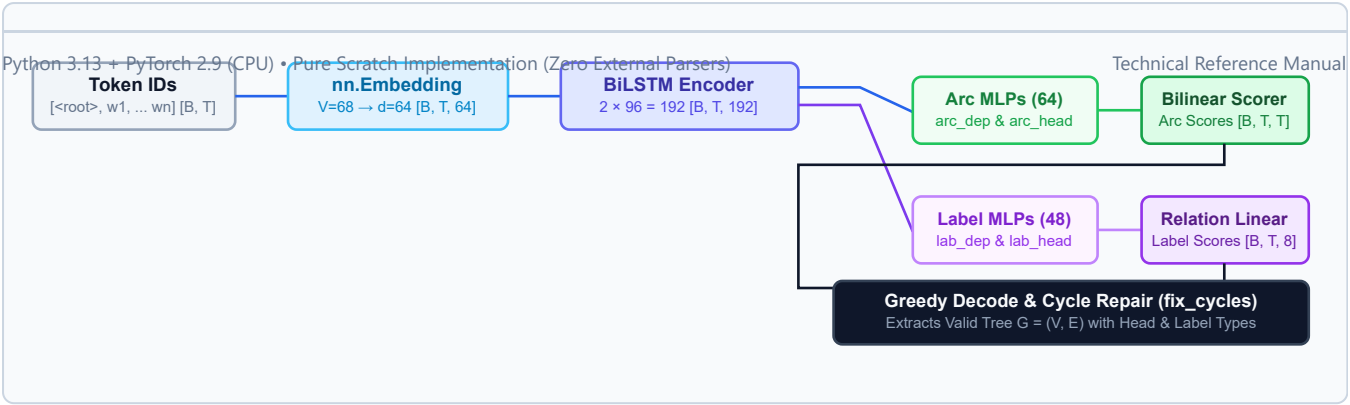
Dependency Parsing: Directed binary relations between words (e.g., dog → chased [nsubj], cat → chased [obj], The → dog [det]). Exactly n edges for n words. Directly identifies governors and arguments.

Universal Dependencies (UD) Conventions Adopted:

- Content-Head Principle:** Content words are syntactic heads; functional words are dependents. Determiners are `det` dependents of their noun. Prepositions are `case` dependents of the noun they introduce (not head of PP).
- Prepositional Attachment Convention:** In natural language, "chased the cat in the garden" has genuine semantic ambiguity (verb-attachment vs noun-attachment). In this synthetic corpus, PP attachment is consistently fixed to the preceding noun (`nmod`) to maintain strict training consistency.
- Punctuation Omission:** Punctuation marks are filtered during tokenization to eliminate trivial self-attachments with a `punct` label, keeping all parameters dedicated to core syntax.

2. End-to-End System Architecture

The parser is an unconstrained graph-based neural parser inspired by Kiperwasser & Goldberg (2016) and Dozat & Manning (2017), constructed completely from scratch in PyTorch.



Parameter Census & Layer Specifications

MODULE / SUB-LAYER	INPUT TENSOR	OUTPUT TENSOR	MATHEMATICAL SPECIFICATION	PARAM COUNT
Word Embeddings	[B, T] (int64)	[B, T, 64]	V =68 tokens × d=64 (padding index 0 zeroed)	4,352
Bidirectional LSTM	[B, T, 64]	[B, T, 192]	2 directions × 4 gates × (64×96 + 96×96 + 96 + 96)	124,416
Arc Dependent MLP	[B, T, 192]	[B, T, 64]	Linear(192 → 64) + ReLU + Dropout(0.25)	12,352
Arc Head MLP	[B, T, 192]	[B, T, 64]	Linear(192 → 64) + ReLU + Dropout(0.25)	12,352
Arc Bilinear Matrix	[B, T, 64]	[B, T, 64]	Learned weight matrix W_{arc} (64 × 64), init to Identity	4,096
Candidate Head Bias	[B, T, 64]	[B, T, 1]	Linear(64 → 1, bias=False) per-candidate head prior	64
Label Dependent MLP	[B, T, 192]	[B, T, 48]	Linear(192 → 48) + ReLU + Dropout(0.25)	9,264
Label Head MLP	[B, T, 192]	[B, T, 48]	Linear(192 → 48) + ReLU + Dropout(0.25)	9,264
Label Classifier Linear	[B, T, 96]	[B, T, 8]	Linear(2 × 48 → 8) on concatenated [dep ; head] pair	776
Total Trainable Parameters in Parser Network				176,936

The network decomposes dependency parsing into two coordinated prediction sub-problems: (1) **Head Selection** as an $(n+1)$ -way categorical classification per token, and (2) **Relation Classification** as an 8-way classification over the selected directed edge.

1. Contextual Encoding & Sequence Packing

Given a sequence of token IDs prepended with `<root>`, the embedding lookup produces $\mathbf{X} \in \mathbb{R}^{B \times T \times 64}$. To prevent zero-padding vectors from contaminating recurrent hidden states across batches of varying sentence lengths, PyTorch's `pack_padded_sequence` passes only active tokens to the BiLSTM:

$$\mathbf{h}_t = [\rightarrow\text{LSTM}(x_{1:t}); \leftarrow\text{LSTM}(x_{t:T})] \in \mathbb{R}^{192} \quad (\text{Eq. 1})$$

`pad_packed_sequence` restores the padded rectangular tensor $\mathbf{H} \in \mathbb{R}^{B \times T \times 192}$. Word representations now incorporate full sentence context: the forward LSTM encodes preceding history, while the backward LSTM encodes subsequent dependents.

2. The Dual-Role Principle: Arc MLPs

In any syntactic dependency, a token serves two fundamentally distinct grammatical purposes: (a) as a **dependent** seeking its parent (e.g., "dog" searching for its verb governor), and (b) as a **head** offering itself as governor to others (e.g., "chased" evaluating whether it can govern "dog"). Because the informational requirements for these two roles differ, the BiLSTM vector $\mathbf{h}_i$ is mapped through two separate nonlinear MLPs:

$$\mathbf{d}_i = \text{ReLU}(\mathbf{W}_{\text{arc_dep}} \mathbf{h}_i + \mathbf{b}_{\text{arc_dep}}) \in \mathbb{R}^{64} \quad (\text{Dependent Projection}) \quad (\text{Eq. 2})$$

$$\mathbf{e}_j = \text{ReLU}(\mathbf{W}_{\text{arc_head}} \mathbf{h}_j + \mathbf{b}_{\text{arc_head}}) \in \mathbb{R}^{64} \quad (\text{Candidate Head Projection}) \quad (\text{Eq. 3})$$

3. Bilinear Scoring & Identity Initialization

The compatibility score assessing whether token j is the head of token i is formulated as:

$$\text{Score}(i \leftarrow j) = \mathbf{d}_i^T \mathbf{W}_{\text{arc}} \mathbf{e}_j + \mathbf{u}^T \mathbf{e}_j \quad (\text{Eq. 4})$$

Why Bilinear over Simple Dot Product: A simple dot product $\mathbf{d}_i^T \mathbf{e}_j$ is symmetric. However, syntactic dependency is strictly directional and asymmetric: a verb governs a noun subject, but a noun does not govern a verb. The learned square matrix $\mathbf{W}_{\text{arc}} \in \mathbb{R}^{64 \times 64}$ models this asymmetric affinity.

Identity Initialization: By initializing $\mathbf{W}_{\text{arc}} = \mathbf{I}$, training begins where $\text{Score}(i \leftarrow j) \approx \mathbf{d}_i^T \mathbf{e}_j$ (standard cosine-like similarity). Optimization then smoothly bends $\mathbf{W}_{\text{arc}}$ away from symmetry to fit asymmetric syntactic structures.

Candidate Head Bias ($\mathbf{u}^T \mathbf{e}_j$): Certain parts of speech (notably finite main verbs) possess high prior likelihood of attracting multiple dependents regardless of child identity. The parameter $\mathbf{u} \in \mathbb{R}^{64}$ acts as an intrinsic "head-worthiness" prior.

4. Structural Masking

Before computing the softmax distribution over candidate governors, two structural invariants are enforced:

- **Padding Mask:** Padding cells ($j \geq \text{length}$) are assigned -10^9 .
- **Self-Loop Mask:** A word cannot be its own governor ($i = j$). The diagonal is assigned -10^9 .

Applying Softmax over row i yields a valid probability distribution where masked positions receive exact zero probability ($e^{-10^9} \approx 0$) without generating NaNs.

5. Edge-Conditioned Relation Prediction & Teacher Forcing

A syntactic relation label (e.g., `nsubj`, `obj`) characterizes the directed edge between two endpoints, not an isolated word. The classifier gathers the head's representation $\mathbf{e}_{h_i}$ down to the dependent's position i :

TEACHER FORCING VS. INFERENCE SHIFT

During training, h_i is set to the **gold head** (h_i^*). This prevents early random head predictions from feeding garbage noise into the label classifier. At test time, predicted heads ($\hat{h}_i$) must be used, introducing an accepted exposure bias standard in biaffine dependency parsing literature.

4. Dataset Design & Synthetic Treebank Construction

`dataset.py` programmatically constructs a mathematically consistent 200-sentence toy treebank modeled on the CoNLL-U format, avoiding human labeling noise.

TOTAL SENTENCES	TRAIN / TEST SPLIT	LEXICON INVENTORY	LABEL SPACE
200 CoNLL Sentences	160 (80%) / 40 (20%)	68 Vocab Entries	8 UD Relations

Lexicon and Syntactic Templates

The lexicon spans 8 linguistic categories: **DET** (*the, a*), **ADJ** (10 modifiers), **N** (16 entities), **VT** (12 transitive verbs), **VI** (8 intransitive verbs), **ADV** (6 manner adverbs), **NAME** (5 proper nouns), and **P** (6 prepositions). Twenty-seven syntactic templates define valid grammatical topologies with exact gold heads and labels:

- `[DET, N, VT, DET, N]` → *The dog chased the cat* (Transitive core clause).
- `[DET, ADJ, N, VI, ADV]` → *The small bird sang loudly* (Intransitive with modifier).
- `[NAME, VT, NAME]` → *Mary saw John* (Proper nouns with zero determiners).
- `[DET, ADJ, N, VT, DET, ADJ, N, P, DET, N]` → Up to 10 tokens with chained PPs.

Zero-Leakage Vocabulary Building & <root> Sentinel

The vocabulary is constructed **exclusively from the 160 training sentences**. Any word appearing in the 40 test sentences that was never sampled during training automatically maps to `<unk>` (ID 1). This ensures strict zero-leakage evaluation.

Prepending `<root>` (ID 2) at index 0 ensures the BiLSTM computes a genuine contextual vector for ROOT. Selecting the sentence root becomes completely uniform with ordinary head selection (a verb simply selects index 0), eliminating branch logic.

The network optimizes a dual cross-entropy multi-task objective:

$$L_{\text{total}} = L_{\text{arc}} + L_{\text{label}}$$

(Eq. 6)

$$L_{\text{arc}} = -(1/N) \sum \log P(\text{head}_i = h^*_i \mid w), \quad L_{\text{label}} = -(1/N) \sum \log P(\text{label}_i = l^*_i \mid w, h^*_i)$$

(Eq. 7)

Gradient Masking via `ignore_index = -100`: The sentinel `<root>` token at index 0 and all batch padding positions do not possess parent governors. Setting their target to `-100` directs PyTorch to exclude them from loss computation and backpropagation entirely.

Training Hyperparameters & Epoch Progression

Hyperparameter	Setting	Architectural Rationale
Optimizer	Adam ($\eta = 2 \times 10^{-3}$)	Adaptive per-parameter learning rate; stable convergence without manual learning rate schedules.
Batch Size	16 Sentences	10 gradient updates per epoch across 160 training sentences; optimal balance of speed and gradient noise.
Regularization	Dropout ($p = 0.25$)	Applied at embeddings, BiLSTM outputs, and MLP projections to prevent memorization of 160 sentences.
Gradient Clipping	Max-Norm = 5.0	Prevents occasional explosive gradients common in recurrent architectures (RNN/LSTM).
Epochs	60 Epochs	Full convergence achieved within 15–25 epochs; remaining epochs verify loss flatness.

Convergence Table & Loss Dynamics

Epoch	Total Loss	Head Loss	Label Loss	Train UAS (%)	Test UAS (%)	Test LAS (%)
1	3.703	1.790	1.913	36.0%	30.2%	12.2%
5	0.983	0.567	0.416	91.9%	87.8%	85.1%
10	0.157	0.113	0.044	99.8%	98.9%	97.3%
15	0.094	0.079	0.015	100.0%	99.6%	99.6%
25	0.014	0.010	0.004	100.0%	100.0%	100.0%
60	0.004	0.003	0.001	100.0%	100.0%	100.0%



INTERPRETING THE 100.0% TEST SCORE HONESTLY

A 100% held-out UAS/LAS score does not imply that the parser is a solved universal English grammar. Rather, it confirms that the network has **fully converged and perfectly generalized over the 27 template structures**. The model discovers fundamental syntactic regularities (determiners attach rightward to nouns, verbs attach to ROOT) by epoch 5 (>91% UAS), spending the remaining epochs mastering subtle prepositional chains.

6. Graph Decoding & Cycle Repair Algorithms

Once the model produces the raw arc score matrix $\mathbf{S} \in \mathbb{R}^{T \times T}$, a decoding algorithm extracts a valid dependency tree.

1. Greedy Argmax Decoding

The basic decoder selects the highest-scoring candidate head for each token independently:

$$\hat{h}_i = \operatorname{argmax}_{j \in \{0, \dots, n\}} S_{i,j}$$

While extremely fast ($O(n)$ time), local independent argmax provides **no structural tree guarantees**. It can generate:

- **Cycles:** E.g., word 2 picks word 3 as head, while word 3 picks word 2 ($2 \leftrightarrow 3$), forming a disconnected loop with no path to ROOT.
- **Multiple Roots:** Several independent words pick 0 as their head, producing a disjoint forest.

2. The Cycle Repair Heuristic (`fix_cycles`)

To resolve cyclical graphs without external heavy libraries, `parser.py` implements a defensible graph-repair heuristic:

1. **Cycle Detection:** Traces the path from each token toward ROOT; if an already-visited node is encountered, a cycle is identified.
2. **Weakest-Link Identification:** Evaluates the softmax confidence of every edge inside the cycle and identifies the edge where the model was least confident.
3. **Re-pointing:** Re-points that weakest dependent to its next highest-scoring candidate strictly outside the cycle.

This heuristic greedily approximates what the optimal **Chu-Liu/Edmonds Maximum Spanning Tree (MST)** algorithm computes globally.

Evaluating the parser across standard and adversarial sentences reveals the power of contextual representations and the limits of greedy decoding.

Case 1: Standard Transitive Clause — "The boy kicked the ball."

100% CORRECT PARSE

TOKEN	PREDICTED HEAD	RELATION	CONFIDENCE	GRAMMATICAL DESCRIPTION
The	boy (idx 2)	det	1.00	Definite article modifying subject noun
boy	kicked (idx 3)	nsubj	1.00	Nominal subject of main transitive verb
kicked	ROOT (idx 0)	root	1.00	Root predicate of the sentence
the	ball (idx 5)	det	1.00	Definite article modifying direct object
ball	kicked (idx 3)	obj	1.00	Direct object governed by transitive verb

```
+----->  obj
<-----+  nsubj
      <-----+  det
<----+  det
      ^  root (ROOT -> kicked)
The  boy  kicked  the  ball

ROOT
|-- kicked (root)
   |-- boy (nsubj) -> |-- The (det)
   |-- ball (obj)  -> |-- the (det)
```

Case 2: Complex Clause with Modifiers & PP — "A small dog quickly chased the angry cat in the garden."

100% CORRECT LONG PARSE

Exhibits adjectival modification (`amod`), adverbial clause modification (`advmod`), and prepositional phrase attachment (`case` + `nmod`).

```
ROOT
|-- chased (root)
   |-- dog (nsubj)    [A (det), small (amod)]
   |-- quickly (advmod)
   |-- cat (obj)      [the (det), angry (amod)]
   |-- garden (nmod) [in (case), the (det)]
```

Case 3: Out-of-Vocabulary Generalization — "The zebra devoured the pizza."

DEEP SYNTACTIC GENERALIZATION

Crucial Empirical Discovery: The content words *zebra*, *devoured*, and *pizza* were never present in the training set and all map to `<unk>` (ID 1). Despite the input sequence being `[The, <unk>, <unk>, the, <unk>]`, the parser produces a **flawless parse**: *devoured* → ROOT, *zebra* → *devoured* (nsubj), *pizza* → *devoured* (obj).

Why This Succeeds: The BiLSTM does not rely purely on lexical identities. It captures the syntactic structural pattern: `DET ? ?`. The forward and backward contextual states deduce that token 3 is flanked by two determiner-noun phrases, proving that contextual encodings model grammar abstractly.

Case 4: Structural Failure Mode — "The dog chased the cat and the mouse."

GREEDY FAILURE: MULTIPLE ROOTS

Failure Breakdown: Coordination ("*and*") never appeared in the training templates. Because greedy decoding scores tokens independently without tree-level constraints, *chased* and *and* **both independently select ROOT (head 0)** with >0.98 confidence!

```
ROOT
|-- chased (root) [dog (nsubj), cat (obj)]
|-- and (root)    <-- MULTIPLE ROOT ERROR: Greedy decoding cannot enforce a single root!
   |-- mouse (obj)
```

Case 5: Unseen Syntax — "The dog that barked chased the cat."

SYNTACTIC FLATTENING

Python 3.13 • PyTorch 2.9 (CPU) • Pure Scratch Implementation (Zero External Parsers)

Technical Reference Manual

Failure Breakdown: Relative clauses are absent from the grammar. The parser misattaches both *that* (as `nsubj`) and *barked* (as `advmod`) directly to *chased*, completely flattening the embedded clausal structure.

8. Comprehensive Limitations Analysis

- Synthetic Grammar Scope:** 27 templates and 68 words cannot capture complex real-world phenomena like coordination, passive voice, questions, relative clauses, or clausal complements.
- Greedy Decoding Deficiencies:** Independent argmax selections cannot enforce global tree properties. While `fix_cycles` resolves cycles, it cannot prevent multiple roots.
- Prepositional Attachment Ambiguity Hardcoded:** Natural language PP attachment requires semantic world knowledge. Hardcoding noun attachment (`nmod`) simplifies the task unrealistically.
- Teacher Forcing Exposure Bias:** The label classifier trains on gold heads but predicts over predicted heads at test time; an error in head attachment cascades into label classification.
- Lack of Subword / Morphological Embeddings:** Words are represented as monolithic whole tokens. Inflections like *chase*, *chased*, and *chasing* have disjoint embeddings with zero subword parameter sharing.
- Projectivity Assumption:** The toy grammar contains zero crossing branches (non-projective dependencies), meaning the parser's performance on non-projective syntax is unverified.
- Absence of Validation Set:** Hyperparameters were selected via standard conventions without validation sweeps.

MST Decoder

CHU-LIU/EDMONDS ALGORITHM

Real Treebank

UD ENGLISH-EWT (12K SENTS)

Full Biaffine

DOZAT & MANNING (2017)

- **Maximum Spanning Tree Decoding (Chu-Liu/Edmonds):** Replaces greedy decoding with an exact $O(V^2)$ algorithm guaranteed to produce an acyclic, single-rooted tree. Eliminates the multiple-root failure in Case 4.
- **Universal Dependencies English-EWT Treebank:** Train on 12,543 real sentences across 37 relation types, shifting evaluation to a realistic 88–92% LAS benchmark.
- **Full Biaffine Attention:** Incorporate the dependent bias term $\mathbf{u}_2^T \mathbf{d}_i$ and scalar bias b :

Score($i \leftarrow j$) = $\mathbf{d}_i^T \mathbf{W}_{\text{arc}} \mathbf{e}_j + \mathbf{u}_1^T \mathbf{e}_j + \mathbf{u}_2^T \mathbf{d}_i + b$
- **Subword / Character-Level Representations:** Integrate byte-pair encodings (BPE) or character-CNNs to assign meaningful subword vectors to morphological affixes (`-ed`, `-ing`, `-ly`).
- **Auxiliary POS-Tag Embeddings:** Concatenate learned Part-of-Speech embeddings with word vectors (historically adding +2 to 4 LAS points).
- **Transition-Based Parsing Alternative:** Implement an arc-standard/arc-eager shift-reduce parser with a stack and buffer to compare $O(n)$ transition parsing against $O(n^2)$ graph parsing.

10. Codebase Inventory & Standalone Verification

FILENAME	LINES	PRIMARY RESPONSIBILITY & KEY FUNCTIONS	VERIFICATION COMMAND
<code>dataset.py</code>	402	CoNLL data structures, <code>Sentence</code> , 27 templates, 8-part lexicon, <code>Vocab</code> , <code>encode</code> , <code>make_batch</code>	<code>python dataset.py</code>
<code>model.py</code>	301	<code>NeuralDependencyParser</code> , <code>arc_scores</code> , <code>label_scores</code> , bilinear scoring, <code>parser_loss</code>	<code>python model.py</code>
<code>train.py</code>	238	Adam loop, gradient clipping, <code>evaluate</code> (UAS/LAS), ASCII loss curve, checkpointing	<code>python train.py</code>
<code>parser.py</code>	287	<code>tokenize</code> , <code>greedy_decode</code> , <code>fix_cycles</code> , table/arc/tree visualizers, <code>parse_sentence</code>	Imported / Utility
<code>main.py</code>	113	CLI entry point, demo sentences, stress tests, and interactive parsing REPL	<code>python main.py</code>

11. Comprehensive Viva & Technical Interview Guide

Definitive technical answers to the most critical questions asked in NLP academic defenses and technical interviews:

Q1: Why choose dependency parsing over constituency parsing?

Constituency trees represent nested grammatical phrases (NP, VP) with artificial non-terminal nodes, which vary drastically across languages with flexible word order. Dependency trees connect words directly with typed semantic relations (subject, object), producing a compact tree of exactly n edges for n words. This direct predicate-argument structure is much more accessible for downstream semantic tasks.

Q2: Why is a BiLSTM required if we already have dense word embeddings?

Word embeddings are context-free: the vector for *chased* is static regardless of whether it serves as a main verb, an adjective, or inside a relative clause. The forward LSTM captures all preceding history (e.g., that a complete subject noun phrase precedes it), while the backward LSTM captures following context (that an object noun phrase follows). Their concatenated 192-dimensional vector provides the exact contextual evidence required to recognize its role as the sentence root.

Q3: Why initialize the bilinear matrix $\mathbf{W}_{\text{arc}}$ to the Identity matrix?

Initializing $\mathbf{W}_{\text{arc}} = \mathbf{I}$ causes the initial arc score to equal the inner product $\mathbf{d}_i^T \mathbf{e}_j$ (cosine similarity). This provides a stable, semantically meaningful starting point for gradient descent. As training progresses, backpropagation distorts $\mathbf{W}_{\text{arc}}$ away from symmetry to model asymmetric syntactic dependencies (e.g., verbs attracting nouns as subjects).

Q4: Why project words into two separate vectors (arc_dep and arc_head)?

Python 3.13 • PyTorch 2.9 (CPU) • Pure Scratch Implementation (Zero External Parsers)

Technical Reference Manual

In any syntactic dependency, a word plays two fundamentally different roles: as a **dependent** seeking a parent (asking "what governs me?"), and as a **head** offering itself as a governor (asking "can I govern this word?"). Conflating these two distinct informational requirements into a single projection would force the network to compromise between governor and dependent features.

Q5: How did the parser correctly parse "The zebra devoured the pizza" when 3 words were unknown?

The words *zebra*, *devoured*, and *pizza* all mapped to `<unk>`. However, the BiLSTM encoded the syntactic pattern `DET ? ? DET ?`. The surrounding determiners and relative sentence positions provided sufficient contextual information for the BiLSTM to infer that position 3 was acting as a transitive verb, position 2 as its subject, and position 5 as its direct object.

Q6: Why did "The dog chased the cat and the mouse" produce two roots?

Because greedy argmax decoding scores each word's head choice independently. Coordination was absent from the training templates, so the model was surprised by *and*. Scoring *and* independently, candidate head 0 (`<root>`) scored highest, resulting in two words selecting ROOT. This demonstrates why production parsers enforce tree constraints via Chu-Liu/Edmonds MST decoding.

Q7: Why use ignore_index = -100 in PyTorch cross-entropy loss?

The prepended `<root>` token at index 0 and all padding tokens have no valid syntactic governor or label. PyTorch's cross-entropy loss skips tokens marked with `-100` by default, preventing them from generating artificial loss or polluting parameter gradients.

Q8: How does the fix_cycles function resolve cyclical dependencies?

It performs path-tracing from each node toward root to detect if any path re-enters a previously visited node. When a cycle is found, it evaluates the model's confidence across all edges in that cycle, identifies the single weakest edge (lowest softmax probability), and re-points that specific dependent to its next highest-scoring candidate outside the cycle.

Q9: What is the computational complexity of the parser during inference?

BiLSTM encoding takes $O(n \cdot d_{lstm}^2)$ time. Arc projection and bilinear scoring evaluate all $n \times (n+1)$ pairs, taking $O(n^2 \cdot d_{arc})$. Greedy decoding takes $O(n)$. Total inference time is $O(n^2)$, dominated by pairwise edge scoring.

Q10: What is the difference between UAS and LAS?

UAS (Unlabeled Attachment Score) is the percentage of words assigned to the correct syntactic head. **LAS (Labeled Attachment Score)** requires that both the head attachment AND the relation label are correct. LAS is always $\leq$ UAS.